

# Collective-Aware Router Microarchitecture in gem5 Garnet

## Tree multicast and congestion-aware express-link bypass

Chunyu Liu and Boyan Pu

September 6, 2026

### Abstract

Lab 4 Topic 3 asks for two router mechanisms: bypass links that skip intermediate Routers and multicast that sends one packet to several destinations. We implement both as physical Garnet datapaths rather than traffic-generator approximations. Tree multicast carries a destination bitmap, replicates only at branch Routers, and reduces internal-link flits by 39.51% on average across 162 paired cases. Its atomic fanout nevertheless causes ten throughput regressions, exposing a concrete scheduling tradeoff. For bypass, a 6,144-pair source-only screen shows that hop count alone is misleading: distance-scaled stride links are neutral at  $0.999\times$  latency speedup. A cycle-aware multi-hop route oracle plus conservative congestion admission raises the same stride design to  $1.430\times$  geometric-mean speedup and 21.87% higher throughput over 1,536 pairs; the median is  $1.082\times$  and the worst case is  $0.909\times$ . As an additional Topic 4 case study, multi-flit tensor streaming shortens a scaled eight-H100 trace window by  $4.728\times$  without changing source or Router flit counts. The report separates these claims, states their baselines and units, and retains negative cases rather than reporting only favorable workloads.

## 1 Introduction

Modern training gives the network a repeated synchronization contract. NCCL exposes collectives such as broadcast and all-reduce across GPU ranks [3]; communication therefore contains structure that ordinary destination-by-destination unicast does not expose. This project asks three questions, with the first two directly matching Topic 3:

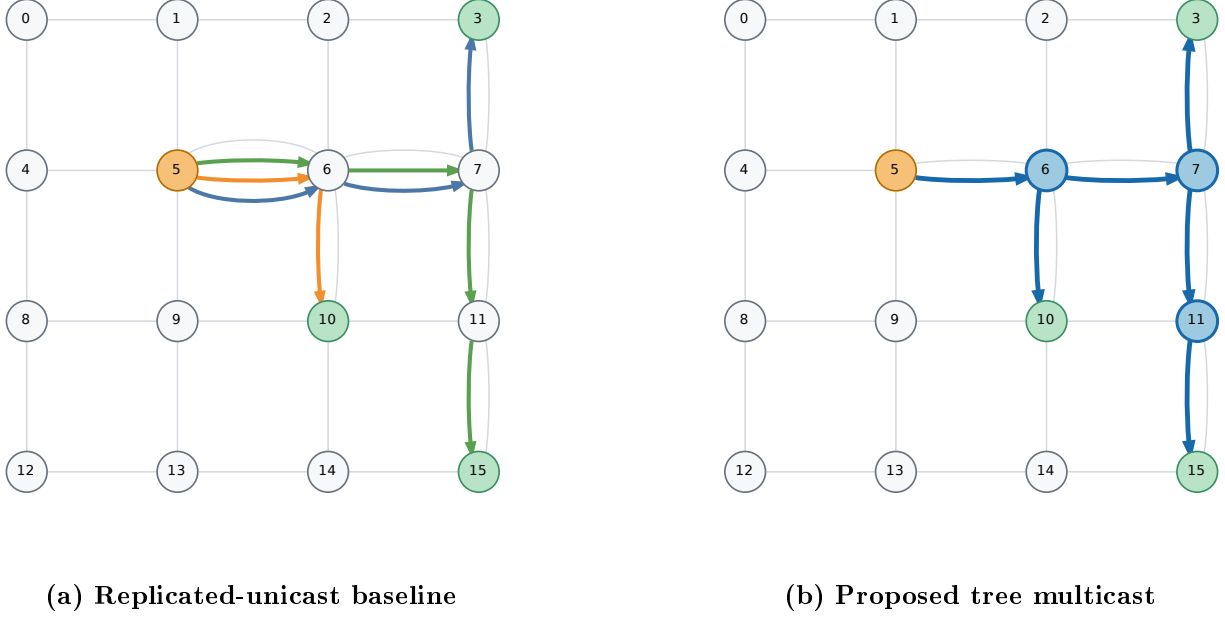
1. Can Router-side replication reduce physical multicast traffic without changing the delivered destination set?
2. Do express links still improve latency after their span-dependent wire delay, added radix, and congestion are counted?
3. As an extension, can a traced all-reduce remain a multi-flit request instead of thousands of serialized scalar operations?

Our answer is deliberately not “all mechanisms always win.” Multicast has a robust traffic benefit but an atomic-fanout throughput cost in a small set of cases. Bypass improves loaded structured traffic, but its source-only stride form fails once wire delay is modeled. That failure motivates the multi-hop and pressure-aware refinement rather than being hidden. Tensor all-reduce is kept as a separate Topic 4 extension so that it cannot inflate the evidence for the two required Topic 3 mechanisms.

The implementation has three corresponding parts:

- **Router-level tree multicast** shares common route prefixes and replicates a single destination-bitmap packet at branch Routers.
- **Physical express-link bypass** adds shortcut links and uses an offline route table to select cycle-saving express hops; the refined stride design may use several monotonic hops and applies congestion-aware admission at every eligible Router.
- **Tensor all-reduce replay** keeps a measured collective as a multi-flit request, reduces lanes in Routers, and broadcasts the result.

The evaluation follows the same structure. Multicast is compared with replicated unicast on the same Mesh. Bypass is compared with plain Mesh XY under identical packet streams and seeds. The broad source-only screen is the primary topology experiment; the multi-hop experiment is a like-for-like refinement prompted by its neutral stride result. Each claim uses its native unit: link-flit reduction, paired latency ratio, throughput change, or trace window.



**Figure 1:** Illustrative 4×4 multicast baseline and proposed path.

Section 2 defines the mechanisms, baselines, and metrics; Section 3 maps them to Garnet components and records the correctness contract. Section 4 evaluates multicast and express-link bypass under controlled synthetic traffic, and Section 5 replays the measured H100 collective trace. The final sections record interpretation limits, authorship, and reproducibility.

## 2 Mechanisms and experimental setup

The implementation exposes two Topic 3 mechanisms and one Topic 4 extension. The baseline and metric definitions below fix the comparison contract before the mechanism descriptions and quantitative results.

### 2.1 Experimental setup: variants and baselines

The implementation uses gem5 v23.0.0.1 [1] (base revision 77fcf26d57) and deterministic XY routing on the cycle-level Garnet network model [2]. The source-only bypass screen was rerun at project revision 135768001a97. The multi-hop run began at 1e8764cde7; its manifest hashes match the pressure-aware sources later committed as ea4c3c9b0f. Thus the recorded source snapshot, not only the branch name at launch, identifies the evaluated binary. A reported ratio always compares the same topology, traffic, packet sequence, payload, request schedule, and seed. The multicast comparison uses replicated unicast as its baseline: the baseline injects one complete packet for each destination, while the proposed path injects one destination-bitmap packet and replicates it at branch Routers. Both variants use the same Mesh, and every selected destination must receive a complete packet. The bypass comparison uses plain Mesh XY as its baseline; the proposed Mesh\_Bypass adds physical express links and an audited route table, while injection, workload, and ordered-VNet behavior remain unchanged.

The 4×4 diagrams below are illustrative mechanism examples, not the full topology scope. The main bypass screen covers both 4×4 and 8×8 meshes, the H100 replay uses a 2×4 mesh, and the multicast performance matrix uses the 4×4 setup shown in Figure 1.

Figure 1 visualizes the multicast comparison before the metric definitions below.

For a paired latency  $L$ , internal-link flit count  $F$ , and throughput  $Q$ , we use

$$S_L = L_{\text{base}}/L_{\text{new}}, \quad R_F = 1 - F_{\text{new}}/F_{\text{base}}, \quad I_Q = Q_{\text{new}}/Q_{\text{base}} - 1.$$

Thus  $S_L > 1$  is faster,  $R_F > 0$  is less link traffic, and  $I_Q < 0$  is a throughput regression. One multicast request completes only after every selected destination receives the complete packet;  $Q$  counts these logical requests per cycle, so both variants perform the same application-level work. We use arithmetic means for additive changes and geometric

**Table 1:** Main controlled-study configurations.

| Parameter   | Multicast matrix                                 | Bypass screen                                          |
|-------------|--------------------------------------------------|--------------------------------------------------------|
| Topology    | 4×4 Mesh, source Router 5                        | 4×4 and 8×8 Mesh                                       |
| Traffic     | 4/8/16 destinations; optional uniform background | uniform, transpose, bit complement, hotspot            |
| Packet size | 1/4/16 flits                                     | 1/4/16/64 flits                                        |
| Load points | 0.1/0.5/1.0, background 0/0.1                    | 16 points from 0.01 to 3.0                             |
| Seeds       | 1, 7, 17                                         | 1, 7, 17                                               |
| Measurement | 20 warm-up, 100 measured, 20 cooldown requests   | 1,000 warm-up, 5,000 measurement, 100,000 drain cycles |

means for latency ratios. The latter is essential for bypass because near-saturation ratios have a long positive tail. The released CSV files retain full distributions, extrema, and regression cases.

Table 1 collects the parameters that otherwise tend to be hidden in command lines. Each comparison is paired by workload and seed.

## 2.2 Tree multicast

The baseline Network Interface injects one physical packet per destination. The multicast path injects one packet carrying a 64-bit destination bitmap. Each Router removes destinations already served, computes the required output branches, and makes a local copy only where selected. Head, body, and tail state is retained per branch. Fanout is atomic: a flit advances only when all selected output VCs have credit. This preserves exact multi-flit delivery, but it also couples a fast branch to a blocked branch—an important interpretation of the throughput results below. The bitmap bounds one collective domain to 64 Routers.

## 2.3 Offline route-table express-link bypass

`Mesh_Bypass` appends bidirectional diagonal or stride links to the ordinary Mesh. Before simulation, a route-table builder enumerates every current-Router/destination pair. The original screen permits one source express hop followed by XY. The refinement lets dimension-aligned stride links be selected repeatedly in the X phase and then the Y phase, but only when the complete suffix has lower modeled cycle cost. The builder constructs the full paths and channel-dependency graph and rejects any placement whose graph is cyclic.

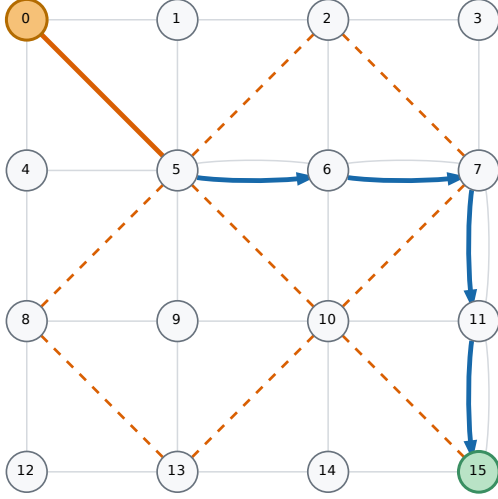
- **Optimistic:** every express link costs one cycle, an upper bound on the value of shortening a path.
- **Distance-scaled:** latency and serialization scale with the Manhattan span, a more conservative proxy for a physical wire.

The cost of an ordinary edge is link latency plus Router latency. An express edge uses its configured wire latency plus one Router traversal and is admitted statically only if it lowers the full suffix cost. Ordered Vnets retain this deterministic table. On unordered traffic, the refined selector may fall back to the monotonic XY edge when the express output or landing output is blocked, the express output is materially more congested, or the destination is detected as a sustained hotspot.

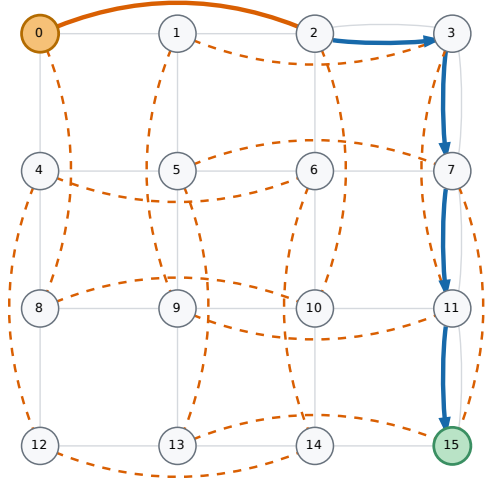
1. Each Router consults the table entry indexed by its own ID and the destination.
2. A stride express entry must move monotonically toward the destination in the current XY dimension and may not overshoot.
3. Both adaptive choices therefore remain in X-before-Y order.
4. A full all-pairs channel-dependency DAG certifies the resulting order.

The bypass screen uses data VNet 2 (four buffers per VC). The express-link cost is either optimistic (one cycle) or distance-scaled by the link’s Manhattan span; the latter is the hardware-relevant model used for the main comparison.

The bypass comparison therefore has a plain Mesh XY baseline and a `Mesh_Bypass` variant with physical shortcut candidates. Figure 2 shows the two candidate placements. The diagonal design retains the original source-only policy; the stride refinement can select multiple aligned express hops along the deterministic dimension-ordered path.



(a) Diagonal placement



(b) Stride-2 placement

**Figure 2:** Illustrative 4×4 bypass placements and route-table-selected paths.

**Legend:** orange dashed lines are installed bidirectional bypass links; solid orange is the selected express hop or hops; solid blue is the ordinary DOR segment; gray lines are ordinary Mesh links.

**Table 2:** Implementation map in gem5 Garnet.

| Mechanism | Main components                                                                        | Added state and behavior                                                                                                     |
|-----------|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Multicast | <code>flit</code> , <code>NI</code> , <code>Router</code>                              | 64-bit destination mask, per-packet branch VCs, Router-side replication, exact completion tracking                           |
| Bypass    | <code>Mesh_Bypass.py</code> , <code>bypass_oracle.py</code> , <code>RoutingUnit</code> | Physical express ports, all-pairs per-Router next-hop table, monotonic DOR paths, channel-dependency audit, runtime fallback |
| Tensor    | <code>NI</code> , <code>Router</code> , replay compiler                                | Per-flit lane ID, per-(collective, lane) accumulator, queued backpressure-safe reduce/broadcast forwarding                   |

### 3 Implementation and validation

The implementation changes the packet metadata, Network Interface (NI), Router fast path, routing unit, and topology construction rather than emulating the mechanisms in the traffic generator. Table 2 summarizes the division of responsibility.

#### 3.1 Atomic multicast fanout

The head flit first computes the pruned-tree children selected by its bitmap. The Router commits branch state only if every required output has a free VC; body and tail flits reuse those VCs. Listing 1 is an excerpt from the implemented two-phase allocation. This makes a multi-flit packet exact and prevents partial branch allocation, but also explains the measured head-of-line coupling: one blocked branch stalls the whole fanout.

**Listing 1:** Source excerpt: atomic multicast branch allocation (Router.cc).

```
const int vnet = head_flit->get_vnet();
for (const int destination : destinations) {
    const int output = collectiveOutputForChild(destination);
    if (output < 0 || !m_output_unit[output]->has_free_vc(vnet)) {
        m_network_ptr->recordMulticastCreditStall();
        return false;
    }
}
for (const int destination : destinations) {
    const int output = collectiveOutputForChild(destination);
    const int outvc = m_output_unit[output]->select_free_vc(vnet);
    fatal_if(outvc < 0, "Router %d lost multicast VC allocation", m_id);
    m_multicast_branches.push_back({output, outvc, destination});
}
```

## 3.2 Auditable bypass routing

Topology construction creates two directed Garnet links for every installed physical shortcut and scales their latency by Manhattan span in the distance-scaled model. Listing 2 shows the implemented dynamic programming recurrence: it compares complete ordinary and express suffixes, and accepts an express edge only when `dor_express` proves monotonic X-before-Y progress. Exact-cost ties prefer the ordinary edge, so a bypass must save a modeled cycle rather than merely a graph hop.

**Listing 2:** Source excerpt: cycle-aware multi-hop DOR selection (bypass\_oracle.py).

```
@lru_cache(maxsize=None)
def best_stride_step(current, destination):
    if current == destination:
        return 0, None
    ordinary_next = xy_step(current, destination)
    suffix_cost, _ = best_stride_step(ordinary_next, destination)
    choices = [(ordinary_edge_cost + suffix_cost, 0,
                ordinary_next, "", None)]
    for link in outgoing[current]:
        if not dor_express(link, destination):
            continue
        suffix_cost, _ = best_stride_step(link["dst"], destination)
        cost = int(link["latency"]) + router_latency + suffix_cost
        choices.append((cost, 1, link["dst"], link["name"], link))
    cost, _, _, _, selected = min(choices)
    return cost, selected
```

The builder then materializes every source–destination path, adds each successive channel pair to a dependency graph, and requires a topological ordering over the entire graph. This is stronger than checking that each individual path terminates: a cycle formed across several routes also fails construction. Runtime adaptation can choose only the table’s express edge or the monotonic XY edge in the same DOR phase, so it does not introduce a new channel class or a dimension reversal.

The performance refinement is completed in `RoutingUnit.cc`, not in the offline oracle alone. After computing local VC/credit pressure and the landing Router’s oracle-selected next output, the retained policy executes the fallback in Listing 3. A sustained many-to-one destination, a blocked express/escape output, or a materially healthier XY output rejects the shortcut. Long packets additionally require actual credit headroom on both the express and landing outputs.

**Listing 3:** Source excerpt: conservative runtime bypass admission (RoutingUnit.cc).

```
if (network->bypassDestinationSkewed(route.dest_router) ||
    express_free == 0 || escape_free == 0 ||
    (long_packet && lookahead_free == 0) ||
    express_free + 1 < xy_free)
    return xy_outputport;

const int landing_capacity = vcs_per_vnet *
    network->getBuffersPerDataVC();
if (long_packet &&
    ((xy_free == vcs_per_vnet && xy_credits == landing_capacity) ||
     express_free < xy_free || express_credits < xy_credits ||
     lookahead_free < vcs_per_vnet ||
     lookahead_credits < landing_capacity))
    return xy_outputport;
```

**Table 3:** Executed validation coverage in the accepted snapshot.

| Gate                      | Coverage         | Required invariant                                        |
|---------------------------|------------------|-----------------------------------------------------------|
| Multicast matrix          | 208 paired cases | Exact bitmap, flit order, no missing/duplicate delivery   |
| Bypass screen/refinement  | 10,752 runs      | Complete table, route termination, acyclic CDG            |
| Multicast $\times$ bypass | 624 runs         | 416 paired interactions complete without route corruption |
| Tensor functional         | 14 cases         | Exact lane count and reduced value on varied meshes       |
| Tensor backpressure       | 32 cases         | Completion under varied link/router latency and occupancy |
| Trace tools               | 12 unit tests    | Schema, sizes, ordering, IDs, and participant validation  |

### 3.3 Streaming tensor reduction

Tensor packets retain their multi-flit boundary while each flit carries a lane identifier. Listing 4 is the Router accumulation point. An entry is released as soon as every local/child contribution arrives; completed lanes wait in a FIFO when their next output VC is unavailable, so backpressure does not discard reduction state.

**Listing 4:** Abridged source excerpt: lane-wise tensor accumulation (`Router.cc`).

```

const int lane = t_flit->get_lane_id();
const auto key = std::make_pair(t_flit->get_collective_id(), lane);
CollectiveLaneState& state = m_collective_lanes[key];
state.sum += t_flit->get_value();
++state.count;
if (state.count == m_collective_expected_fanin) {
    const int64_t sum = state.sum;
    m_collective_lanes.erase(key);
    if (m_collective_root)
        forwardCollectiveLane(sum, lane, CollectiveOp::Broadcast,
                               m_id, t_flit);
    else
        // parent is derived from m_collective_parent_outport
        forwardCollectiveLane(sum, lane, CollectiveOp::Reduce,
                               parent, t_flit);
}

```

### 3.4 Validation contract

Correctness gates check payload values and delivery sets, not only simulator termination. The compact-data builder additionally rejects incomplete pairs, non-finite metrics, inconsistent route metadata, and wrong run counts before generating any headline figure.

## 4 Evaluation results

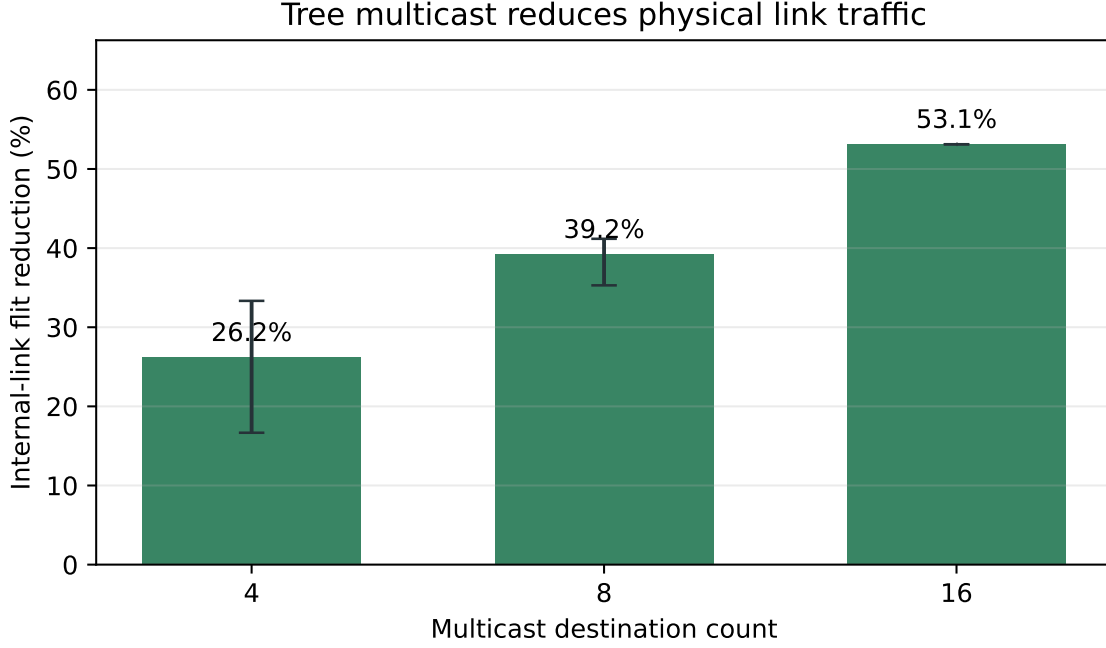
### 4.1 Multicast performance

Tree multicast removes physical link work reliably, but its throughput benefit depends on branch synchronization. The results below separate those two claims rather than treating traffic reduction as a proxy for throughput.

Across 162 paired cases, tree multicast reduces internal-link flits by 39.51% on average (median 41.18%, range 16.67–53.13%). The effect is monotone in fanout: 26.19% for four destinations, 39.22% for eight, and 53.13% for sixteen. This is the cleanest evidence for the mechanism because both variants use the same Mesh and the metric counts only physical link flits. Figure 3 summarizes the reduction by fanout.

Latency also improves in this matrix: geometric-mean speedup is 3.638 $\times$  (median 3.588 $\times$ , range 1.200–22.512 $\times$ ), and all 162 paired values exceed one. The multicast implementation uses a dedicated Router fast path, so its switch-allocation and crossbar sequence differs from unicast. We therefore read latency as an implemented-system result and use link flits for the structural comparison.

Lower link traffic does not always translate into higher throughput. The arithmetic mean change is +190.89%, but the distribution is highly skewed by full-group traffic; ten of 162 cases regress, all at four destinations, with a worst case of –18.24%. The paired distribution is shown in Figure 4. Atomic fanout explains how both statements can be true: the tree does less total link work, yet a single blocked branch can hold up all copies.



**Figure 3:** Internal-link flit reduction from tree replication; bars are means and whiskers show the observed range for each fanout.

## 4.2 Multicast sensitivity and tradeoff

The workload breakdown makes the dependency concrete. At 4, 8, and 16 destinations, mean throughput changes are +34.38%, +154.87%, and +383.42%, but the four-destination group contains all ten regressions. By packet size, the mean changes are +261.65% (1 flit), +169.45% (4 flits), and +141.57% (16 flits); the negative cases concentrate in the longer, loaded four-way requests. Figure 5 shows the same pattern together with the min-max spread, while Figure 6 relates traffic reduction to throughput on a case-by-case basis.

**Multicast takeaway.** Tree replication reliably removes duplicate physical work, and the saving grows with fanout. Throughput follows a different workload curve because atomic fanout and the dedicated fast path introduce their own scheduling effects.

## 4.3 Primary experiment: offline route-table bypass

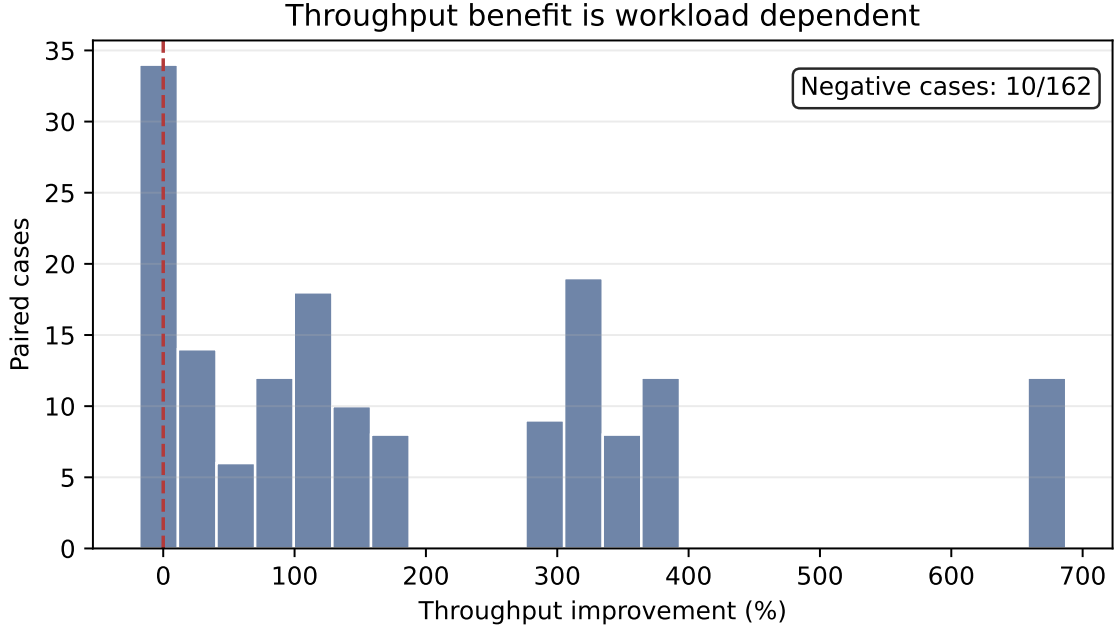
Express-link bypass is useful only when path shortening survives wire cost and contention. The screen below tests that claim across topology, traffic, and wire-model choices.

The bypass sweep uses four standard synthetic traffic patterns. In `uniform_random`, each source chooses a destination uniformly at random. In `transpose`, a source at mesh coordinate  $(x, y)$  sends to  $(y, x)$ , creating a structured permutation. In `bit_complement`, it sends to the mirrored coordinate  $(W - 1 - x, H - 1 - y)$ , pairing opposite locations in the mesh. In `hotspot`, 50% of packets target one fixed Router (Router  $N/2$  in an  $N$ -node mesh) and the rest use uniform random destinations. These four patterns isolate route structure and contention.

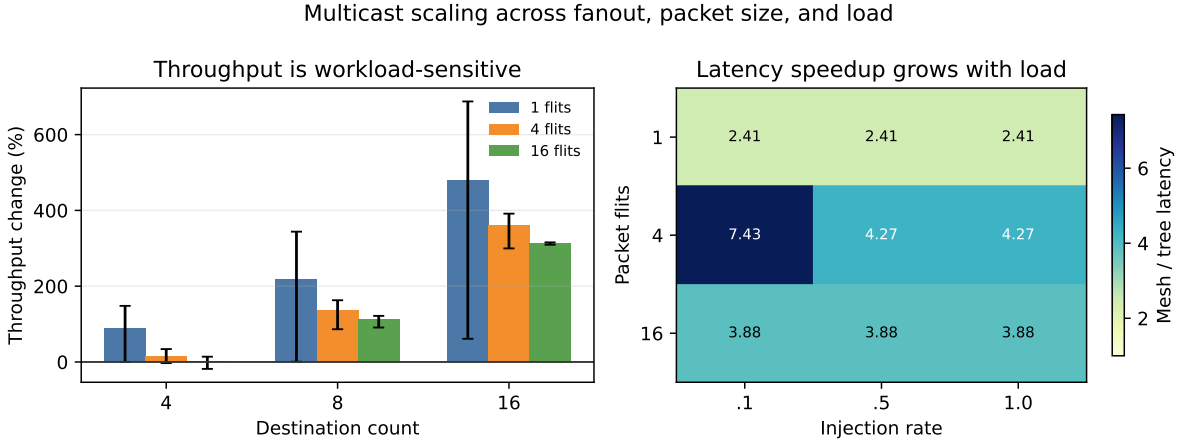
The original offline route-table sweep compares the same synthetic workloads and seeds on Mesh XY and on each physical express-link design, with the route table selecting at most one source express hop. The latency column in Table 4 is the geometric mean over all 1,536 paired seed cases per design; additive changes use arithmetic means. Reporting the median and regression tail later is necessary because saturated baseline points can produce very large ratios. The fixed table isolates the value of an admissible shortcut, while the regression count exposes how often the aggregate result fails to transfer to an individual workload.

The cost records explain the design choice. A  $4 \times 4$  diagonal placement adds 9 undirected links (wire proxy 18, maximum radix 8), while stride adds 16 (wire proxy 32, maximum radix 6); on  $8 \times 8$  the corresponding link counts are 49 and 96. Physical topology choice therefore combines hop reduction with wire length, radix, and buffer cost.

Figure 7 adds the multi-hop refinement to the four source-only designs.



**Figure 4:** Paired multicast throughput changes, including the high-fanout tail and the ten regression cases.

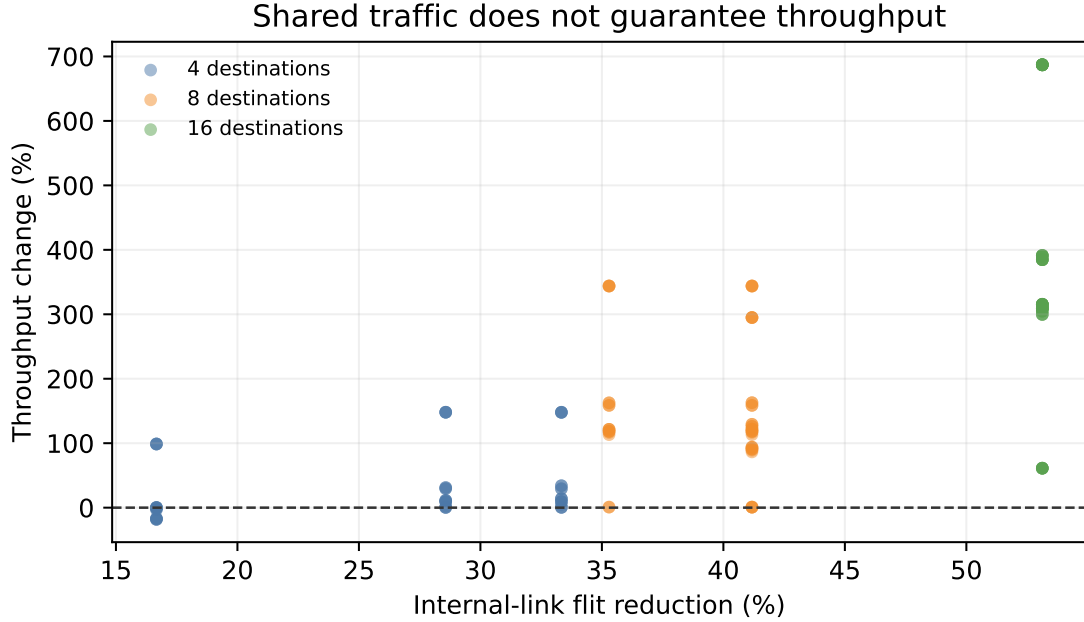


**Figure 5:** Sensitivity of multicast throughput and latency to fanout, packet size, and offered load.

In the source-only screen, diagonal links provide the strongest latency result, while stride links remove more Router traversals but incur a larger physical-link budget. At offered load  $\leq 0.1$ , however, the distance-scaled geometric means are  $0.999\times$  for diagonal and  $0.993\times$  for stride. The gain therefore comes from favorable loaded cases rather than a universal reduction in base latency. The optimistic wire model is a sensitivity bound; distance-scaled latency is the hardware-relevant comparison.

#### 4.4 Multi-hop stride refinement

The neutral source-only stride result identifies a structural limit: a packet can skip only one Router even when its path crosses several aligned shortcuts. The refined policy consults the cycle-aware table at every Router, so a packet may use several stride links while remaining monotonic in X and then Y. For unordered data traffic, conservative admission retains the static shortcut unless the express or landing output is blocked, the express output is materially more congested than XY, or epoch counters identify the destination as a sustained hotspot. For packets above 32 flits, the selector also checks credit headroom and the oracle-selected output at the landing Router, admitting the shortcut only when XY is under pressure and the landing output is idle. A 96-pair pilot rejected a more aggressive version despite its higher mean because it introduced four regressions larger than 5%; the retained guard had none in



**Figure 6:** Traffic reduction versus throughput change for individual paired cases; the dashed line marks unchanged throughput.

**Table 4:** Original source-only topology and wire-model screen over 1,536 paired seed cases per design; latency ratios use geometric means.

| Design                    | Latency speedup | Regressions | Throughput change | Router-work change |
|---------------------------|-----------------|-------------|-------------------|--------------------|
| Diagonal, distance-scaled | 1.131×          | 350/1,536   | +9.41%            | +5.49%             |
| Diagonal, optimistic      | 1.148×          | 164/1,536   | +9.44%            | +5.53%             |
| Stride, distance-scaled   | 0.999×          | 539/1,536   | +2.72%            | −11.35%            |
| Stride, optimistic        | 1.024×          | 321/1,536   | +2.81%            | −11.26%            |

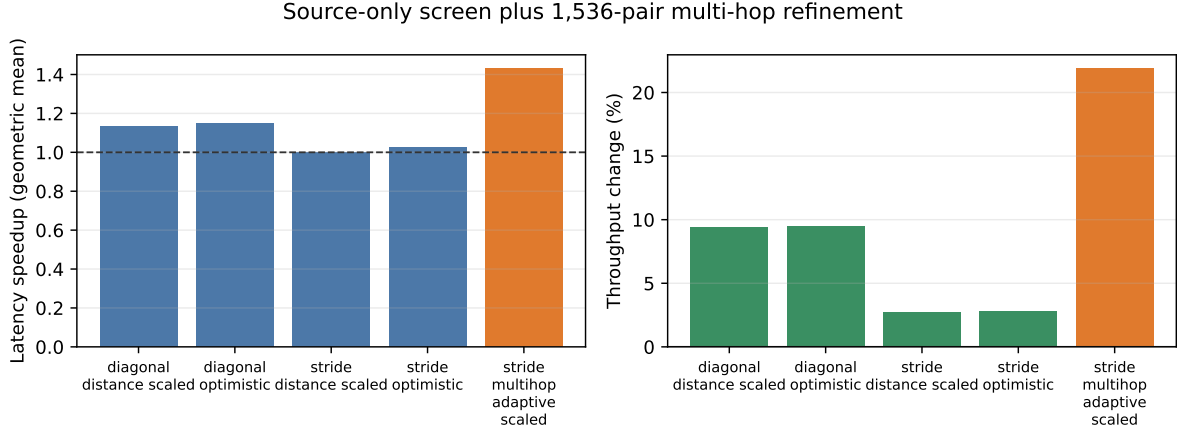
Router-work change is  $F_{\text{new}}/F_{\text{base}} - 1$  for each run's *aggregate* Router traversals. Positive throughput can therefore produce a positive traversal count even when individual paths shorten; negative values mean less Router work. The throughput and Router-work columns must therefore be read together.

the same pilot.

Across all 1,536 pairs, the median is 1.082× and 243 ratios are below one, but only one regresses by more than 5%. The best ratio, 97.94×, comes from a near-saturated baseline and is why we do not report an arithmetic mean as the headline. The worst is 0.909× for 4×4, 16-flit hotspot traffic at offered load 0.3. At offered load at most 0.1 the geometric mean is 1.051×; above 0.7 it is 1.627×. The 64-flit group now reaches 1.134× with no regression larger than 5%. Thus the 1.430× headline is not a claim of universal base-latency improvement: multi-hop bypass is most valuable when the skipped Router stages also relieve loaded paths, while hotspot admission largely restores the baseline on many-to-one traffic.

We also tested a natural follow-up rather than silently tuning around that worst case. On the same 96-pair pilot, disabling bypass only within Manhattan radius 1, 2, or 3 of a detected hotspot produced overall geometric means of 1.6830×, 1.6838×, and 1.6837×, all below the retained global guard's 1.6844×. More importantly, the targeted worst case was 0.9001×, 0.9090×, and 0.9090×, below the predeclared 0.95× acceptance threshold. We therefore rejected all three before a full run. This negative pilot is retained in `report/data/bypass_hotspot_radius_pilot.csv` and is not mixed into the 1,536-pair headline result.

**Bypass takeaway.** Express links are useful when route structure makes a saved hop valuable, while distance-scaled wires and extra radix impose real costs. Moving from one source-only shortcut to audited multi-hop DOR raises geometric-mean speedup from 0.999× to 1.430× for stride links; congestion admission reduces the severe-regression tail without weakening the wire model.



**Figure 7:** Latency speedup and throughput change for the source-only screen and the distance-scaled multi-hop adaptive refinement.

**Table 5:** Distance-scaled multi-hop stride refinement. Latency is a geometric mean; throughput and Router-traversal-per-delivered-flit changes are arithmetic means. The last column counts regressions larger than 5%.

| Group          | Pairs | Speedup | Throughput | Router/flit reduction | > 5% regress. |
|----------------|-------|---------|------------|-----------------------|---------------|
| Overall        | 1,536 | 1.430×  | +21.87%    | 18.10%                | 1             |
| 4×4            | 768   | 1.291×  | +4.37%     | 17.37%                | 1             |
| 8×8            | 768   | 1.583×  | +39.37%    | 18.84%                | 0             |
| Uniform random | 384   | 1.842×  | +19.63%    | 24.98%                | 0             |
| Transpose      | 384   | 1.182×  | +5.50%     | 15.92%                | 0             |
| Bit complement | 384   | 1.892×  | +61.07%    | 14.54%                | 0             |
| Hotspot        | 384   | 1.015×  | +1.28%     | 16.99%                | 1             |

## 4.5 Multicast-bypass interaction

A 624-run factorial gate combines both multicast modes with Mesh XY and the bypass topology. All 416 Mesh/bypass pairs complete with exact destination sets and unchanged wire-distance accounting. Table 6 shows that shortcut use remains mechanism-dependent: replicated unicast can route each copy independently, whereas tree multicast preserves its pruned XY branch semantics. This gate establishes composability; it is not substituted for the broader unicast bypass screen above.

## 5 H100 trace case study

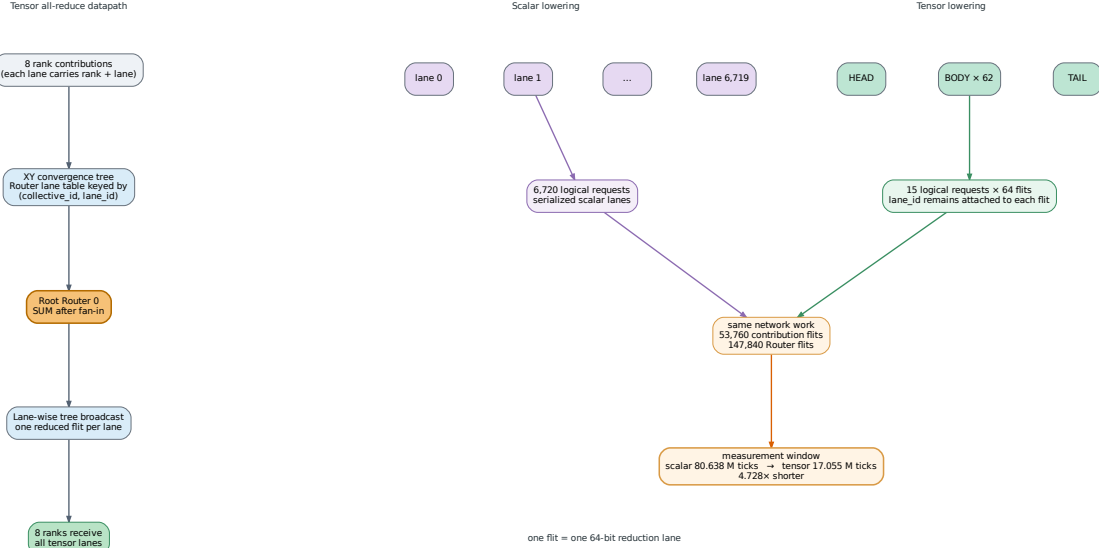
This additional Topic 4 case study preserves a measured collective’s ordering while testing request representation. Its scaled tick counts remain separate from the factorial Topic 3 aggregates.

### 5.1 Trace lowering and implementation

The input is an executed eight-H100 80GB HBM3 NCCL microbenchmark (PyTorch 2.8.0+cu128, CUDA 12.8, NCCL 2.27.3). The replay compiler selects the 15 measurement-phase broadcasts and 15 all-reduces, represents communication with 16-byte flits, and scales payload bytes and relative release times by 1/1024. This preserves event ordering

**Table 6:** Multicast-bypass interaction over 104 pairs per row.

| Multicast mode     | Wire model      | Link-flit reduction | Latency regressions |
|--------------------|-----------------|---------------------|---------------------|
| Replicated unicast | Distance-scaled | 16.10%              | 39/104              |
| Replicated unicast | Optimistic      | 16.10%              | 0/104               |
| Tree multicast     | Distance-scaled | 8.33%               | 15/104              |
| Tree multicast     | Optimistic      | 8.33%               | 0/104               |



**Figure 8:** Tensor all-reduce datapath and the scalar-lane and tensor-request replay lowerings.

while making the trace practical for cycle-level replay.

For broadcast, each selected source event is lowered to variable-length tree-multicast packets. For all-reduce, the scalar replay creates independent serialized lanes; each lane converges rank values in the Routers and then travels back down the convergence tree. The tensor replay keeps each event as one multi-flit logical request, tracks a lane ID per flit, and performs the same Router-side reduction and tree broadcast while retaining the request’s multi-flit boundary. The scalar and tensor paths are distinct collective implementations from the bitmap-based `tree_multicast` path; all three exploit shared tree prefixes.

Figure 8 connects the datapath to the replay representation. It also makes clear why the tensor comparison preserves the same contribution and Router-flit counts: only the logical request boundaries and completion scheduling change.

The replay compares Mesh XY with a four-link, distance-scaled diagonal bypass arm. Every route selected in this  $2 \times 4$  placement is ordinary XY, making the bypass arm a noninterference control rather than evidence of shortcut speedup.

## 5.2 Replay experiments and results

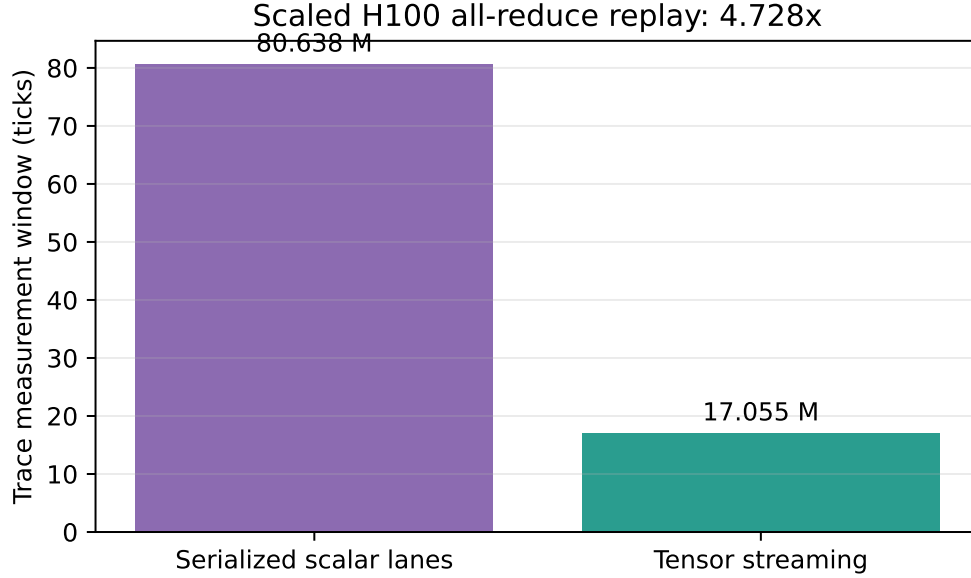
The broadcast replay expands 15 source events into 30 tree-multicast requests; both designs complete 6,720 source flits and 210 destination deliveries in a 17,047,500-tick window. The all-reduce replay compares serialized scalar lanes with multi-flit tensor requests under the same route and release schedule.

Table 7 records the three replay views. Both topology arms are identical, confirming that unused physical links do not perturb the trace.

**Table 7:** Scaled H100 replay summary; p95 values use different logical units and must not be compared across scalar and tensor rows.

| Replay view             | Logical requests | Source flits | Window (ticks) | p95 (ticks) |
|-------------------------|------------------|--------------|----------------|-------------|
| Broadcast               | 30               | 6,720        | 17,047,500     | 643,500     |
| Scalar all-reduce lanes | 6,720            | 53,760       | 80,638,000     | 10,000      |
| Tensor all-reduce       | 15               | 53,760       | 17,054,500     | 2,567,500   |

Both physical designs complete 53,760 exact lane reductions and deliveries with 147,840 Router flits. Tensor-request p50/p95/p99 latency is 647,500/2,567,500/2,567,500 ticks; these values cannot be compared directly with scalar-lane p95 because the logical request units differ.



**Figure 9:** End-to-end replay window for scalar-lane and tensor-request lowerings of the same scaled all-reduce trace.

Scalar lowering records an 80,638,000-tick window, so tensor streaming is  $4.728\times$  shorter on both Mesh XY and Mesh Bypass. Source and Router flit counts are identical, which isolates the measured gain to request representation and completion scheduling while rank contributions and network traffic remain fixed. The result characterizes the configured replay model; translating it to training-step speed requires an end-to-end study.

**Trace-replay takeaway.** Keeping a tensor as a streaming collective removes scalar serialization in this network model while preserving exact arithmetic and delivery invariants. The effect appears in the total trace window; per-request tail percentiles use different logical units for the two lowerings.

## 6 Limitations and interpretation boundary

The experiments establish the mechanisms within the tested model. The scope of the evidence is:

- The multicast matrix covers one  $4\times 4$  source placement (Router 5); corner, edge, and center source placements are future coverage. Its dedicated fast path and atomic fanout expose branch serialization under load.
- Bypass wire cost uses a Garnet latency, serialization, and static-cost proxy. Area, power, repeaters, and timing closure remain outside the model. Refined stride routes may use multiple monotonic express hops; their complete channel-dependency graph is audited before simulation.
- The bypass screen uses synthetic unicast traffic on data VNet 2. The distance-scaled screen is the hardware-oriented comparison, with optimistic timing serving as a sensitivity point.
- The H100 input is an executed collective microbenchmark with  $1/1024$  byte and time scaling. Full model traces are future workload coverage. Broadcast and tensor replay on the shared topology use ordinary XY paths.
- Tensor accumulation uses software state with arithmetic latency and finite table capacity outside the model. Chunk boundaries are validated by the contract and carried by the replay protocol.
- The results characterize cycle-level Garnet mechanisms. NVLink/NVSwitch measurements and end-to-end model throughput are future validation targets.

## 7 Division of labor

Chunyu Liu authored multicast and bypass implementation, correctness checks, trace capture, optimization, and the associated analysis. Boyan Pu authored the tensor all-reduce datapath, replay contract, validator, backpressure

stress cases, and tensor statistics. Both authors integrated branches, reviewed regressions, reran the final quantitative snapshot, and verified this report.

## 8 Reproducibility and artifact trail

Compact accepted snapshots are in `report/data/`; generated vector and preview figures are in `report/figures/`; raw outputs are git-ignored under `report/raw-results/head-77fcf26d/`. The source trace and replay hashes, software versions, scaling semantics, and artifact roots are recorded in `report/data/trace_replay_provenance.txt`. The compact-data builder rejects wrong run counts, non-finite values, failed regression stages, inconsistent VNet or routing metadata, incomplete multicast pairs, and missing designs before writing summary tables or figures.

The original source-only bypass matrix contains 7,680 simulator runs. The accepted multi-hop refinement adds 3,072 runs and stores its exact arguments, simulation windows, and source hashes in `report/data/bypass_multihop_manifest.json`. The resulting 1,536 paired rows are summarized in `report/data/bypass_multihop_summary.csv`; the policy-selection and rejected-radius pilots are retained separately, so they cannot be mistaken for final-matrix samples.

The full command matrix is retained in `report/README.md`. The compact snapshot and every figure can be rebuilt with:

```
python3 tests/gem5/lab4/run_lab4_full_gate.py --jobs 64
python3 report/scripts/build_final_report_data.py
python3 report/scripts/plot_multicast.py
python3 report/scripts/generate_architecture_figures.py
```

The trace-replay runners retain their original four-link diagonal topology as a controlled H100 case-study arm. The new multi-hop stride result belongs to the broad synthetic screen and is not retroactively substituted into that fixed trace placement.

## 9 Conclusion

Both required Topic 3 mechanisms are implemented and evaluated against matched baselines. Tree multicast shares route prefixes and removes 39.51% of internal-link flits on average, rising to 53.13% at fanout 16. Its ten throughput regressions show that atomic branch synchronization remains a real microarchitectural cost. For bypass, the source-only distance-scaled screen is the diagnostic result: diagonal reaches  $1.131\times$ , while stride is neutral at  $0.999\times$  despite reducing Router traversals. Reconsulting an audited cycle-cost oracle at every Router and admitting express hops only under safe local and landing pressure raises stride to  $1.430\times$  geometric-mean speedup and +21.87% throughput. The  $1.082\times$  median,  $0.909\times$  worst case, and rejected radius sweep bound that claim. The additional tensor experiment preserves 15 multi-flit requests and shortens the scaled H100 window by  $4.728\times$  with identical network work.

The common lesson is narrower than “more network features are faster.” Collective structure should be exposed only where it removes measurable physical work; a shortcut should be selected by modeled cycle cost and current pressure rather than hop count; and every speedup should be paired with its distribution, regression tail, and completion contract.

## References

- [1] N. Binkert et al., “The gem5 Simulator,” *ACM SIGARCH Computer Architecture News*, vol. 39, no. 2, 2011, pp. 1–7. doi: [10.1145/2024716.2024718](https://doi.org/10.1145/2024716.2024718).
- [2] N. Agarwal, T. Krishna, L.-S. Peh, and N. K. Jha, “GARNET: A Detailed On-Chip Network Model inside a Full-System Simulator,” *ISPASS*, 2009, pp. 33–42. doi: [10.1109/ISPASS.2009.4919636](https://doi.org/10.1109/ISPASS.2009.4919636).
- [3] NVIDIA, “NVIDIA Collective Communication Library Documentation.” <https://docs.nvidia.com/deeplearning/nccl/user-guide/>.